

Exploratory Data Analysis and Data Balancing

Zidio Development

Week Two Report

Explanation and Findings

Abstract

Think of this report as me sitting next to you, scrolling through the Week 2 notebook, and explaining out loud what's happening on screen cell by cell, in the order it actually happened. Week 1 was about getting clean data out of a database. Week 2 is about actually sitting with that data long enough to understand it: staring at charts, asking "wait, why does that look like that?", and slowly fixing the one big structural problem sitting at the heart of this whole project the fact that fraud is incredibly rare compared to normal activity. For every piece of code in the notebook, I've tried to answer the same five questions a curious teammate would ask: what is this code actually doing, why are we doing it, why are we doing it right now instead of some other point in the process, what did we expect to happen, and once the result came back, what is it actually telling us. Nothing has been left out every cell, every chart, every printed number gets its own honest explanation in plain, everyday language.

Introduction to Week 2

So here's where we stood going into Week 2. Week 1 was pure plumbing work connect to a database, pull the transactions table out, make sure it's clean, done. That's necessary, but it doesn't teach you anything about the data itself. And here's the thing about machine learning that trips a lot of people up when they're starting out: you cannot just hand a model a spreadsheet of numbers and expect it to magically figure out what fraud looks like. The model only ever knows what you show it and how you frame the problem for it. If you skip the step of actually looking at your data first, you're basically driving blind.

That's the whole point of Week 2. Before touching a single machine learning algorithm, we needed to sit down with the data and ask it some honest questions. How rare is fraud, really? Does the size of a purchase say anything about whether it's fake? Does the time of day matter? Are there hidden relationships between columns we should know about before we start building anything? And once we understood all of that, we needed to fix the elephant in the room: our dataset is almost entirely made up of normal transactions, and a model trained carelessly on that will look great on paper while being completely useless in practice.

The notebook behind this week's work is called 02_EDA_Feature_Engineering, and it has twelve cells. I'm going through every single one of them, in the order they run, grouped into the same overall steps as before so the flow feels familiar but this time I'm

not skimming. I'm explaining what each cell is doing, why it exists, why it comes at that particular point in the story, what we thought would happen, and then actually reading the chart or the printed numbers it produced, out loud, the way you'd explain it to a friend looking over your shoulder.

Step 1: Loading the Data and Uncovering the Class Imbalance Challenge

Cell 1:- What is this code?

This is the setup cell, and it doesn't produce anything flashy no chart, no big number but every single thing that happens for the rest of the notebook depends on it running first. It imports the tools we're going to need: Pandas to hold and manipulate the data as a table, Matplotlib and Seaborn to draw charts, and two things from Scikit-Learn and imbalanced-learn StandardScaler and SMOTE that we won't actually touch for a while yet, but it's good practice to import everything up front rather than scattering imports throughout the notebook. It also runs `sns.set_theme(style="whitegrid")`, which just tells Seaborn “hey, give every chart from now on this same clean, light-grid look,” so the whole notebook feels visually consistent instead of every chart looking like it came from a different project.

Then it does the actual work: it opens a connection to the SQLite database file we built back in Week 1 (`fraud_detection.db`), runs a plain SQL query “`SELECT * FROM transactions`” to grab absolutely everything in the transactions table, and loads it straight into a Pandas DataFrame called `df`. As soon as that's done, it closes the database connection, because there's no reason to leave it open once the data is sitting safely in memory. The cell wraps up by showing us the first five rows with `display(df.head())` and printing a structural summary with `df.info()`, which lists every column, how many non-empty values it has, and what data type it is.

Why are we doing this, and why now?

This has to be the very first thing that happens, for the obvious reason that you can't analyze data you haven't loaded yet. But there's a subtler reason too by pulling from the Week 1 database instead of some raw file, we're keeping a clean separation between “data engineering week” and “data science week.” Anything we do from here on, no matter how messy or experimental, can't accidentally corrupt the original database, because we're only ever working with a copy sitting in memory.

What did we expect, and what did we actually get?

We expected to see a well-formed table with a sensible number of rows and columns, and that's exactly what came back: 100,000 transaction rows spread across 44 columns, everything

from raw personal details like first, last, and dob, to the transaction itself (amt, category, trans_time), all the way to behavioural features that had already been engineered back in Week 1, like customer_avg_amount_7_day and merchant_risk_30_day. Nothing was missing df.info() showed every single column with a full 100,000 non-null values, meaning there were no gaps to worry about at this stage.

Cell 2:- What is this code?

Now we get to the first real chart of the week, and it's aimed squarely at the most important column in the entire project: is_fraud. This column only ever holds a 0 or a 1- 0 for a normal, legitimate transaction, and 1 for a confirmed case of fraud. The code builds a bar chart with sns.countplot(data=df, x='is_fraud', palette='Set2'), which is Seaborn's way of saying “count how many rows fall into each category and draw one bar per category.” But it doesn't stop there right after, there's a small loop that goes through every bar on the chart, calculates what percentage of the whole dataset that bar represents, and stamps that percentage as text directly above the bar.

Why are we doing this, and why now?

This is the very first real question we need answered before we do anything else: how rare is fraud in this dataset? Everything downstream how we split the data, whether we need to balance it, how we'll eventually judge a model's performance depends on knowing this number early. If you don't check this before building anything, a model can absolutely trick you into thinking it's brilliant when it's secretly doing nothing useful at all.

What does this graph actually show, and why does it look like that?

Look at the chart: one bar towers over the other. The “normal” bar (0) is enormous, sitting at 98,897 transactions, while the “fraud” bar (1) is so small next to it that it barely registers just 1,103 transactions. The percentage labels above each bar spell it out even more bluntly: 98.9% of everything in this dataset is a normal transaction, and only 1.1% is fraud. That's not a small imbalance, that's an enormous one, and it's the single most important fact we've learned so far. A model that never actually learns anything and just always guesses “normal” would still be right 98.9% of the time which sounds amazing until you realize it would catch zero fraudsters, which is the entire point of the project. This is what data scientists call the Accuracy Paradox, and it's exactly why the rest of Week 2 exists to make sure we don't accidentally build a model that cheats its way to a good-looking score.

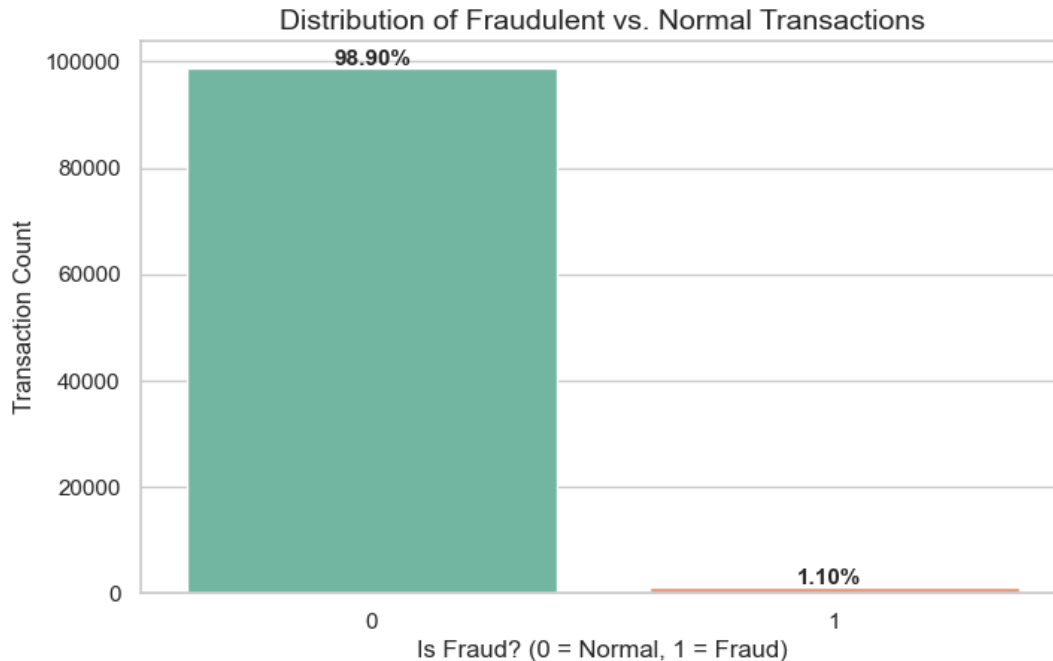


Figure 1: Class Imbalance Bar Chart:- Normal vs. Fraudulent transactions, with the exact percentage stamped above each bar.

Cell 3:- What is this code?

A tiny one-liner: `print(df.columns.tolist())`. That's it, it just prints out the name of every single column in the dataset as a plain list.

Why are we doing this, and why now?

Right after seeing how lopsided the fraud column is, it's a natural moment to pause and take stock of everything else we have to work with, before we start slicing and dicing the data further. It costs nothing to run and it saves a lot of confusion later you don't want to be three steps deep into a preprocessing pipeline and suddenly wonder “wait, is there a column called `trans_time` or `trans_time_hrs`?” This cell answers that up front.

What did we get?

A full list of 44 column names, confirming the dataset has exactly what `df.info()` told us it had a mix of identity fields, transaction details, and a set of engineered behavioral features (things like `merchant_risk_1_day` and `customer_num_trans_7_day`) that will turn out to matter a lot later, once we get to the correlation heatmap and the modeling steps.

Step 2: Transaction Amount Analysis (amt)

With the imbalance problem out in the open, the next obvious question is: does the size of a transaction actually tell us anything about whether it's fraudulent? The notebook answers this

twice once with the full, unfiltered picture on a log scale, and once with a simpler, everyday-range version that's easier to read at a glance. I'll walk through both.

Part A:- The Full Picture on a Log Scale (Cell 4)

What is this code?

This cell draws a boxplot comparing the amt column between normal and fraudulent transactions, using `sns.boxplot(data=df, x='is_fraud', y='amt', hue='is_fraud', palette='pastel', legend=False)`, and immediately switches the Y-axis to a logarithmic scale with `plt.yscale('log')`. Right underneath the chart, it also runs `df.groupby('is_fraud')['amt'].describe()`, which spits out a full statistics table count, mean, standard deviation, minimum, the 25th/50th/75th percentiles, and the maximum calculated separately for the normal group and the fraud group.

Why are we doing this, and why now?

Now that we know how rare fraud is, the natural next question is whether it looks different in some measurable way and the amount of money involved is the most obvious place to start checking. We use a log scale here specifically because transaction amounts in real financial data are wild a \$3 coffee sitting right next to a \$9,000 purchase and a normal chart would squash all the everyday, small-value transactions into an unreadable line at the very bottom while a handful of huge outliers stretch the scale into uselessness.

What does this graph and the numbers under it actually show?

Even squinting at a log scale, you can see the orange fraud box sitting noticeably higher up the chart than the blue normal box its median and its whole interquartile range are shifted upward. The statistics table backs that up with hard numbers: the average fraudulent transaction is \$534.25, versus just \$68.18 for a normal one more than seven times higher. The median tells the same story from a different angle: a typical fraud case is around \$397.55, while a typical normal purchase is only \$44.46. Interestingly, the single largest transaction in the entire dataset (\$14,653.32) was actually a normal one, not fraud which is exactly why we needed the log scale in the first place, otherwise that one outlier would have flattened this entire comparison into a useless smear near the X-axis.



Figure 2: Boxplot of transaction amount by class, log scale, with the full statistical breakdown underneath.

Class	Count	Mean	Std Dev	Min	25%	Median	75%	Max
Normal (0)	98,897	\$68.18	\$159.84	\$1.00	\$9.05	\$44.46	\$81.36	\$14,653.32
Fraud (1)	1,103	\$534.25	\$393.45	\$6.38	\$262.03	\$397.55	\$922.67	\$1,270.12

Part B:- A Cleaner, Everyday-Range View (Cell 10, revisited)

What is this code?

Later in the notebook, this same amount comparison gets revisited, but this time in a friendlier form. The code first filters the dataset down to only transactions under \$1,000, `df_filtered = df[df['amt'] < 1000]` and then draws a plain boxplot of `is_fraud` against `amt` on that filtered slice. No log scale this time, because we've already cut out the handful of extreme values that would have required one.

Why are we doing this, and why now?

The log-scale version earlier is statistically complete, but let's be honest reading a log-scaled axis takes a bit of getting used to. This second version exists purely to answer the same question in a way that's instantly readable for anyone, technical or not: within the range where 99% of purchases actually happen (under \$1,000), do fraudsters spend more, less, or about the same as everyone else?

What does this graph actually show?

Even after removing the extreme outliers, the fraud box still sits clearly higher than the normal box. Normal transactions cluster down near the bottom of the chart with a tight, low range, while fraudulent ones spread higher and wider. In plain terms even sticking to everyday spending amounts, fraud still tends to involve noticeably bigger purchases than normal activity. That's a genuinely useful signal we can hand to a future model.

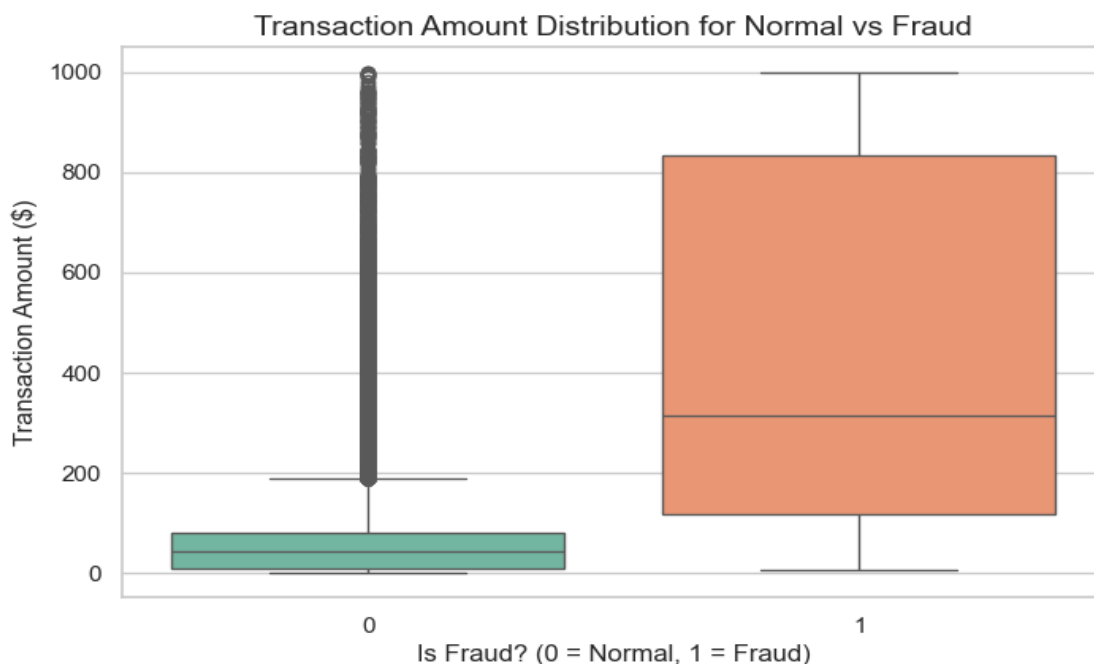


Figure 3: Transaction Amount Distribution for Normal vs Fraud, filtered to transactions under \$1,000.

Step 3: Going Deeper (Age, Density Curves, and Scatter Patterns)

Cell 5:- What is this code?

This cell does two things. First, it builds something that didn't exist in the raw data at all: a customer's age. The dataset gives us a date of birth (dob) and a transaction date (trans_date), but no ready-made "age" column, so the code converts both columns into proper datetime objects with `pd.to_datetime()`, subtracts one from the other to get a difference in days, and divides that by 365 to turn it into a whole number of years: `df['user_age'] = (df['trans_date'] - df['dob']).dt.days // 365`.

Second, once `user_age` exists, the cell builds a two-panel figure side by side. The left panel is a Kernel Density Estimate (KDE) plot of transaction amount, split by fraud status, drawn on a log scale. The right panel is a scatter plot with every single transaction plotted as a dot, with user age on the X-axis, transaction amount on the Y-axis (also log scale), colored by whether that transaction was fraud or not.

Why are we doing this, and why now?

We'd already established that amount seems to matter, using boxplots and summary statistics. But a boxplot flattens a lot of nuance it only shows you the median, the quartiles, and the outliers, not the actual shape of the data. A KDE plot instead draws a smooth curve showing exactly where the data piles up and where it thins out, which is a much richer picture. And since we're already thinking about “what kind of person is likely to be a fraud victim,” it made sense to bring age into the picture at the same time, using a scatter plot to see if fraud clusters in a particular age band or spreads out evenly across everyone.

What does this graph actually show?

On the left, the KDE curves for normal and fraud transactions peak at clearly different spots along the amount axis the fraud curve's peak sits further to the right than the normal curve's, meaning fraud amounts genuinely cluster in a higher range rather than just having a slightly higher average dragged up by a few outliers. On the right, the scatter plot shows fraud points (a distinct color) scattered fairly broadly across almost every age group rather than bunching up around one narrow age range which tells us that, at least on its own, age doesn't look like a strong standalone predictor of fraud the way transaction amount does. That's a useful negative finding too it tells the team not to over-invest in age as a feature by itself.

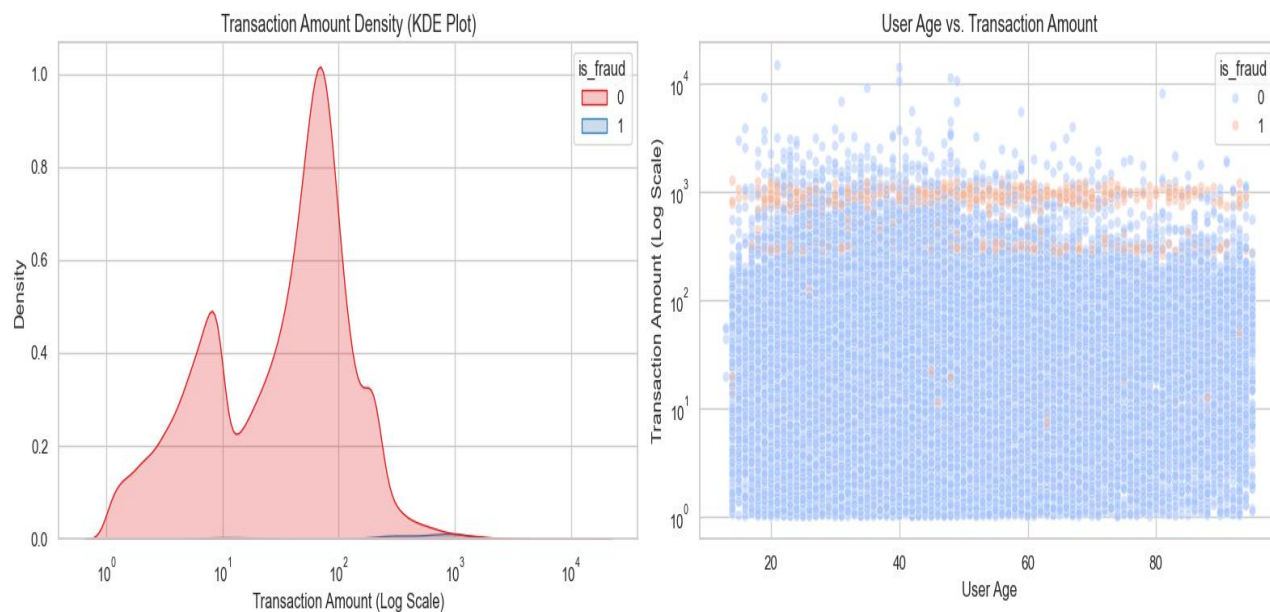


Figure 4: Left KDE density plot of transaction amount by fraud class (log scale). Right Scatter plot of user age vs. transaction amount, colored by fraud status.

Step 4:- Feature Correlation Heatmap

Cell 6:- What is this code?

This cell zooms all the way out and looks at the whole dataset at once instead of one column at a time. It grabs every purely numeric column with `df.select_dtypes(include='number')` deliberately using 'number' rather than listing types by hand, so it catches every flavor of numeric column (int32, int64, float32, float64) without missing any. It then calculates the correlation between every pair of those numeric columns with `numeric_df.corr()`, and draws the whole thing as a big color-coded grid using `sns.heatmap()`, with the actual coefficient number printed inside every single box, and the color scale locked between -1 and +1 so the colors mean the same thing everywhere on the chart.

Why are we doing this, and why now?

Up to this point we'd only been comparing `is_fraud` against one column at a time amount, then age. This cell exists to catch two things a one-at-a-time approach would miss: which numeric columns actually relate to fraud that we haven't checked individually yet, and which columns are basically saying the same thing as each other (redundant features), which is worth knowing before we start feeding everything into a model.

What does this graph actually show?

It's a big grid, and most of it is a pale, washed-out color which is actually reassuring, because it means most feature pairs genuinely don't have a strong relationship with each other, so we're not drowning in redundant information. But scattered through the grid you can spot a few deep red patches clusters where features are strongly, positively related to each other, which makes intuitive sense once you notice they're mostly the merchant risk columns (`merchant_risk_1_day`, `merchant_risk_7_day`, `merchant_risk_30_day`, `merchant_risk_90_day`) sitting next to each other. That tells us those columns are measuring a very similar underlying idea across slightly different time windows, which is exactly the kind of thing worth remembering before deciding which features to actually keep for modeling.

What does this graph actually show?

The bars are not remotely flat across the 24 hours there's a clear, visible bump during the very late-night and early-morning hours, with noticeably fewer fraud cases happening during the middle of a typical waking day. That's not just a nice-to-know fact it's a genuinely strong, usable signal. It means the hour of the day a transaction happens at is something a future model should absolutely pay attention to, because fraudsters really do seem to have a preferred window of opportunity.

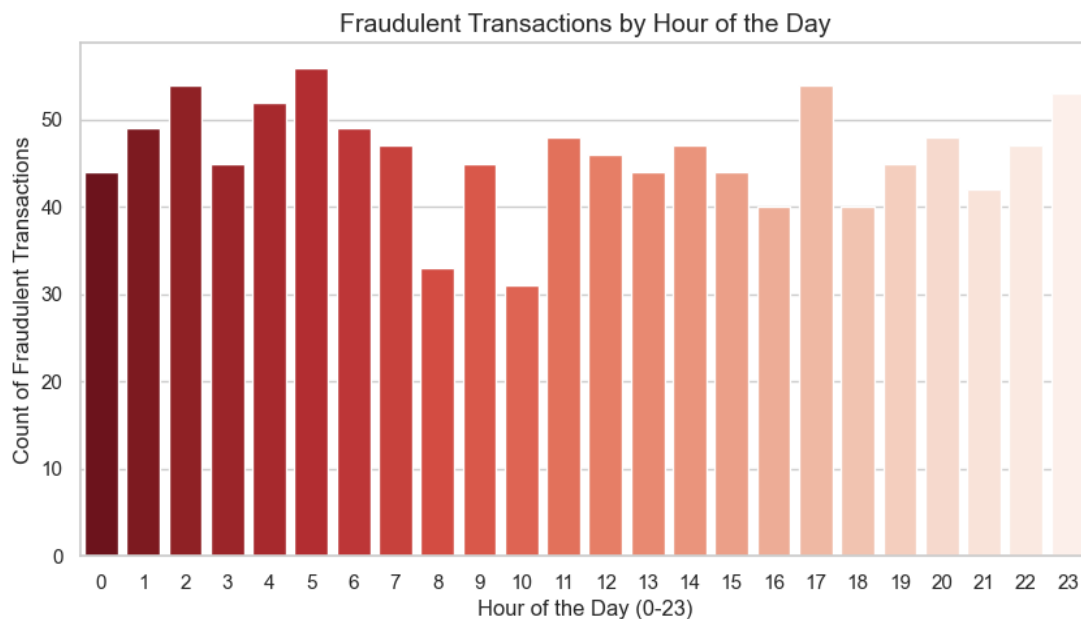


Figure 6: Count plot showing Fraudulent Transactions by Hour of the Day.

Step 6: First Preprocessing Pass Encoding, Scaling, and an 80/20 Split with SMOTE

Before landing on the leaner pipeline that Week 3 actually builds on, the notebook first works through a more elaborate attempt encoding text columns into numbers, scaling continuous columns, and testing things out with an 80/20 train-test split. It's not the version we carried forward, but it's real work that taught us something, so it deserves the same honest explanation as everything else.

Cell 7:- What is this code?

This cell handles two very different jobs at once. First, it decides which text columns are worth turning into numbers and which ones aren't. It picks three low-cardinality columns to keep and encode gender, category, and state meaning columns with only a small, manageable number of distinct values. Everything else that's text-heavy and high-cardinality (ssn, cc_num, first, last, street, city, zip, job, dob, acct_num, profile, trans_num, trans_date, trans_time,

merchant) gets dropped outright to form a leaner `df_clean`. Then it One-Hot Encodes those three chosen columns with `pd.get_dummies()`, which turns each possible category (each U.S. state, for instance) into its own True/False column. Finally, it fits a `StandardScaler` on four continuous columns `amt`, `customer_num_trans_1_day`, `customer_avg_amout_1_day`, and `merchant_risk_1_day` rescaling them so they all sit on a comparable numeric range.

Why are we doing this, and why now?

Now that the exploring is done and we understand the data, it's time to start shaping it into something a machine learning algorithm can actually digest and algorithms only understand numbers, not text. The reason we're picky about which columns to encode is memory: a column like `merchant` or `job` has thousands of unique values, and one-hot encoding something like that would explode into hundreds of thousands of mostly-empty columns, which is a fast way to crash a machine with a `MemoryError`. Scaling matters for a similar reason if one column ranges from 0 to 1 and another ranges into the thousands, some algorithms will unfairly treat the bigger-range column as more important just because of its scale, not because it's actually more informative.

What did we get?

A clean, memory-safe `DataFrame` called `df_encoded` that grew from the original 44 columns up to 91 the extra columns are all the new True/False flags created by one-hot encoding the state and category columns. Printing `df_encoded.head()` and its shape confirmed everything worked exactly as intended, with no memory errors and no leftover text columns.

Cell 8:- What is this code?

With the encoded data ready, this cell splits it into inputs and answers. `X` holds every feature column except `is_fraud`, and `y` holds just the `is_fraud` column itself. Then it performs an 80/20 split using `train_test_split(X, y, test_size=0.20, random_state=42, stratify=y)`, 80,000 rows go into training, 20,000 are set aside for testing.

Why are we doing this, and why now?

This has to happen before we touch `SMOTE`, for a very specific reason: you can't fairly balance and augment data that you haven't already separated into "what the model learns from" and "what we'll judge it on" later otherwise you risk contaminating the test set. The `stratify=y` setting is doing quiet but important work here: it makes sure the rare ~1.1% fraud rate is preserved in roughly the same proportion in both the training set and the test set, instead of randomly dumping most of the fraud examples into just one side by chance.

What did we get?

The printed shapes confirmed a training set of (80000, 90) and a test set of (20000, 90), and the training set's class breakdown came out to 79,118 normal transactions against 882 fraud cases almost exactly the same 98.9%/1.1% ratio as the full dataset, which is exactly what stratify=y is supposed to guarantee.

Cell 9:- What is this code?

This is where SMOTE (Synthetic Minority Over-sampling Technique) makes its first appearance. The code applies it strictly to the training data from the previous cell `smote.fit_resample(X_train, y_train)` producing a newly balanced `X_train_resampled` and `y_train_resampled`, then draws a simple bar chart of the new class counts and prints the before-and-after numbers for comparison.

Why are we doing this, and why now?

We already know from Step 1 that fraud makes up barely more than 1% of the data. Left as-is, a model trained on that would barely see enough fraud examples to learn anything meaningful from them. SMOTE fixes this by generating synthetic, statistically realistic fraud examples based on the ones we do have, until the two classes are balanced and it has to happen after the train-test split, specifically on the training portion only, otherwise the test set would get contaminated with fake data and give us a dishonest picture of how well the model actually performs.

What does this graph and the numbers actually show?

Before SMOTE, the training set was wildly lopsided: 79,118 normal versus just 882 fraud. After SMOTE, the bar chart shows two bars of exactly equal height 79,118 versus 79,118. That's the balance we were after. It's worth saying plainly what this does and doesn't mean: we haven't found 78,236 new real fraud cases out of nowhere we've generated synthetic examples that share the statistical fingerprint of real fraud, purely so the training process has enough signal to actually learn from.

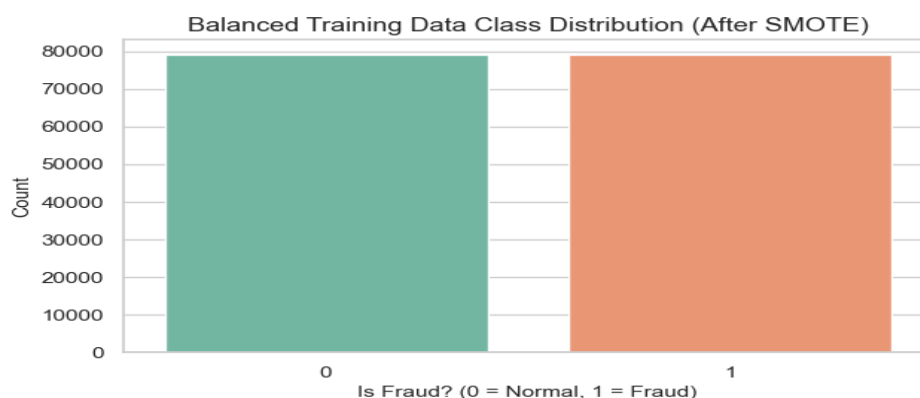


Figure 7: Balanced training data class distribution after SMOTE was applied to the 80/20 split (79,118 vs 79,118).

Step 7: Final Preprocessing for Modeling Dropping IDs, Keeping It Numeric, and a 70/30 Split with SMOTE

Cell 12:- What is this code?

This is the last cell in the notebook, and it's where everything gets pulled together into the cleaner, final pipeline that Week 3 will actually build models on top of. It starts fresh from the original df rather than the heavily one-hot-encoded df_encoded from the previous attempt, and does three things, in order.

- It drops two columns that are basically just ID numbers rather than useful behavior, cc_num and zip with `df_model = df.drop(columns=['cc_num', 'zip'])`.
- It keeps only the purely numeric columns using `select_dtypes(include=['int64', 'float64'])`, since algorithms need numbers, not free text like names or job titles, and separates that into X (the features) and y (the is_fraud target).
- It performs a 70/30 stratified split a slightly bigger test set this time than the 80/20 attempt before it again using `stratify=y` to keep the fraud ratio consistent in both halves, and prints the class counts before doing anything else.
- Then, exactly like before, it initializes SMOTE and applies it only to the training portion X_train and y_train and prints the class counts one more time, before and after, to prove the balancing actually worked.

Why are we doing this, and why now?

Dropping cc_num and zip matters because those columns are essentially unique identifiers, not behavior. If a model is allowed to see a raw credit card number, it risks “learning” that one specific card is fraudulent rather than learning what fraud actually looks like in general, which means it would be useless the moment it saw a card number it had never encountered before. The switch to a 70/30 split (instead of 80/20) gives us a slightly larger, more statistically stable test set to evaluate against later. And SMOTE gets applied one more time, for exactly the same reason as before because roughly 1% fraud simply isn't enough raw examples for a model to learn from without a boost.

What did we get?

Before SMOTE, the 70% training split held 69,228 normal transactions against just 772 fraud cases. After SMOTE, that same training set held exactly 69,228 of each a clean, perfectly even 50/50 split while the 30% test set was never touched, kept completely real and untouched so it can honestly judge whatever model gets trained on this data in Week 3.

The one trap we made absolutely sure to avoid

This is worth calling out clearly because it's the mistake that trips up almost everyone the first time they deal with imbalanced data: applying SMOTE to the whole dataset before splitting it into train and test. It's tempting because it's fewer lines of code, but it quietly contaminates the test set with synthetic examples, and the model ends up looking like it has 99% accuracy purely because it's secretly being tested on data it basically already “knows.” We made sure SMOTE only ever touched `X_train` and `y_train` in both attempts, leaving the test set completely pure for an honest evaluation down the line.

```
Training data shape BEFORE SMOTE:
is_fraud
0      69228
1       772
Name: count, dtype: int64

Training data shape AFTER SMOTE:
is_fraud
0      69228
1      69228
Name: count, dtype: int64
```

Summary of Week 2 Progress

Stepping back and looking at the whole week: we started with a database table and ended up with a dataset we genuinely understand. We know fraud is rare just 1.1% of everything and we know exactly what that means for how we have to approach modeling. We know fraudulent transactions tend to be bigger, both in the full statistical picture and in the everyday spending range. We engineered a brand-new age feature from scratch and learned, honestly, that age alone doesn't look like a strong predictor. We mapped out how every numeric column relates to every other one, and spotted a cluster of merchant-risk features that are closely related to each other. We found a real, usable pattern in when fraud happens during the day. And we tested two different ways of preparing the data for modeling, both landing on the same core principle: balance the training data with SMOTE, but never, ever touch the test data with it.

Every one of these findings points in the same direction transaction amount and time of day both carry real signal about fraud risk, and the severe class imbalance that could have wrecked a naive model has now been properly, honestly handled. Week 3 picks up right where this leaves off: training actual predictive models like Random Forest and XGBoost on this balanced training data, and then judging them fairly against the untouched test set we were so careful to protect.